

Estructura léxica: líneas e indentación, tokens, sentencias y comentarios

Tecnología



Categoría

Introducción

Dificultad

Repositorio

Sistema de indentación y su importancia en la estructura de bloques

Python se distingue de otros lenguajes de programación por su enfoque único en la **legibilidad** del código. Una de las características más distintivas que contribuye a esta legibilidad es su sistema de indentación, que no es solo una convención de estilo, sino una parte fundamental de la **sintaxis** del lenguaje.

A diferencia de lenguajes como Java o C++ que utilizan llaves `{ }` para delimitar bloques de código, Python utiliza la **indentación** (espacios en blanco al principio de una línea) para definir la estructura y jerarquía del código. Esta característica hace que el código Python sea naturalmente más ordenado y consistente en su apariencia.

Reglas básicas de indentación

La indentación en Python sigue algunas reglas específicas:

- Cada nivel de indentación debe ser **consistente** dentro de un mismo bloque de código
- Se recomienda usar **4 espacios** por nivel de indentación (estándar PEP 8)
- No se deben mezclar tabulaciones y espacios (preferiblemente usar solo espacios)
- La primera línea de un archivo no debe estar indentada

Veamos un ejemplo básico de cómo funciona la indentación en una estructura condicional:

```
# Ejemplo de indentación en una estructura if-else
edad = 18

if edad >= 18:
    print("Eres mayor de edad")
    print("Puedes votar")
else:
    print("Eres menor de edad")
    print("No puedes votar aún")

print("Esta línea está fuera de la estructura condicional")
```

En este ejemplo, las líneas indentadas bajo el `if` y el `else` pertenecen a sus respectivos bloques. La última línea, al no estar indentada, se ejecuta independientemente del resultado de la condición.

Bloques anidados

La indentación también permite crear **estructuras anidadas** de forma clara y visual:

```
temperatura = 25
humedad = 80

if temperatura > 20:
    print("Hace calor")
    if humedad > 70:
        print("Y está húmedo")
        print("Recomendación: usar ropa ligera y beber agua")
    else:
        print("Pero no está húmedo")
        print("Recomendación: disfrutar del día")
else:
    print("No hace calor")
```

Cada nivel de anidamiento añade un nivel adicional de indentación (generalmente 4 espacios más), lo que permite visualizar fácilmente la **jerarquía** del código.

Errores comunes de indentación

Los errores de indentación son **errores de sintaxis** en Python y provocarán que el programa no se ejecute. Algunos errores comunes incluyen:

- **IndentationError**: Ocurre cuando la indentación no es consistente o cuando falta indentación donde se requiere.
- **TabError**: Ocurre cuando se mezclan tabulaciones y espacios de manera inconsistente.

Veamos ejemplos de estos errores:

```
# Error de indentación - falta indentación
if True:
print("Este código provocará un IndentationError")

# Error de indentación - indentación inconsistente
if True:
    print("Primera línea correctamente indentada")
print("Esta línea provocará un IndentationError por indentación inconsistente")
```

Indentación en diferentes estructuras

La indentación es esencial en todas las estructuras que definen bloques en Python:

- Estructuras condicionales (if, elif, else)
- Bucles (for, while)
- Definición de funciones (def)
- Clases (class)
- Bloques try-except

Ejemplo con diferentes estructuras:

```
def calcular_promedio(numeros):
    if not numeros:
        return 0

    suma = 0
    for numero in numeros:
        suma += numero
        if numero > 10:
            print(f"Número grande encontrado: {numero}")

    return suma / len(numeros)

# Uso de la función
notas = [7, 8, 12, 9]
```

```
promedio = calcular_promedio(notas)
print(f"El promedio es: {promedio}")
```

Ventajas del sistema de indentación

El sistema de indentación de Python ofrece varias ventajas:

- **Legibilidad mejorada:** El código es naturalmente más ordenado y fácil de leer.
- **Consistencia:** Fuerza a todos los programadores a seguir el mismo estilo de formato.
- **Menos verbosidad:** Elimina la necesidad de caracteres delimitadores como llaves.
- **Prevención de errores:** Hace visibles problemas de estructura que podrían pasar desapercibidos con otros sistemas.

Configuración en Visual Studio Code

Si estás utilizando Visual Studio Code como tu IDE para Python, puedes configurar fácilmente la indentación:

- Visual Studio Code detecta automáticamente que estás trabajando con Python y aplica la **indentación de 4 espacios** por defecto.
- Puedes verificar la configuración actual en la barra de estado (esquina inferior derecha), donde aparece "Espacios: 4".
- Para cambiar la configuración, puedes hacer clic en esta indicación o usar la configuración del editor:
- Presiona `Ctrl+,` (o `Cmd+,` en Mac) para abrir la configuración
- Busca "Python Indentation" o "Editor: Tab Size"

```
# Visual Studio Code mostrará líneas guía verticales para ayudarte
# a visualizar los niveles de indentación
def funcion_ejemplo():
    # Primer nivel de indentación
    for i in range(5):
        # Segundo nivel de indentación
        if i % 2 == 0:
            # Tercer nivel de indentación
            print(f"{i} es par")
```

Recomendaciones para una buena indentación

Para mantener un código limpio y profesional:

- Sigue la guía de estilo **PEP 8** que recomienda 4 espacios por nivel de indentación
- Configura tu editor para que inserte **espacios en lugar de tabulaciones**
- Mantén la **consistencia** en todo tu proyecto
- Evita líneas demasiado largas que requieran muchos niveles de indentación (máximo 3-4 niveles)
- Utiliza la función de **autoindentación** de tu editor (en VS Code, puedes usar `Shift+Alt+F` para formatear el documento)

La indentación en Python no es solo una cuestión estética, sino un componente **estructural** del lenguaje que influye directamente en cómo se ejecuta el código. Dominar este sistema es fundamental para escribir código Python correcto y profesional.

Tokens léxicos: identificadores y palabras clave

Cuando escribimos código en Python, este se divide en unidades fundamentales llamadas **tokens léxicos**, que son los bloques básicos de construcción del lenguaje. Entender estos elementos es esencial para escribir código Python correcto y eficiente.

Los tokens léxicos en Python se pueden clasificar en varios tipos, pero nos centraremos en dos particularmente importantes: los **identificadores** y las **palabras clave**.

Identificadores

Los identificadores son nombres que creamos para identificar variables, funciones, clases, módulos y otros objetos en nuestro código. En Python, los identificadores siguen reglas específicas:

- Deben comenzar con una **letra** (a-z, A-Z) o un **guion bajo** (`_`)
- Después del primer carácter, pueden contener letras, números y guiones bajos
- Son **sensibles a mayúsculas y minúsculas** (nombre, Nombre y NOMBRE son tres identificadores diferentes)
- No pueden ser palabras clave del lenguaje

Ejemplos de identificadores válidos:

```
# Identificadores válidos
nombre = "Ana"
edad_usuario = 25
_contador = 0
miClase = "Python"
```

```
CONSTANTE = 3.14159
variable1 = True
```

Ejemplos de identificadores no válidos:

```
# Estos causarían errores de sintaxis
2variable = 10      # No puede comenzar con un número
mi-variable = 20    # No puede contener guiones
class = "Python"    # No puede ser una palabra clave
```

Convenciones de nomenclatura

Aunque Python permite flexibilidad en la creación de identificadores, existen convenciones de estilo recomendadas en la guía PEP 8:

- **snake_case** para variables, funciones y métodos (palabras en minúsculas separadas por guiones bajos)
- **PascalCase** para nombres de clases (cada palabra comienza con mayúscula sin separadores)
- **MAYÚSCULAS_CON_GUIONES** para constantes

```
# Siguiendo convenciones PEP 8
nombre_completo = "Juan Pérez"    # Variable (snake_case)

def calcular_promedio(numeros):    # Función (snake_case)
    return sum(numeros) / len(numeros)

class UsuarioRegistrado:          # Clase (PascalCase)
    pass

PI = 3.14159                      # Constante (MAYÚSCULAS)
MAX_INTENTOS = 3
```

En Visual Studio Code, estas convenciones son resaltadas por el linter de Python (como Pylint o Flake8) si lo tienes instalado, ayudándote a mantener un estilo consistente.

Palabras clave (keywords)

Las palabras clave son **términos reservados** que tienen un significado especial en Python y no pueden usarse como identificadores. Estas palabras definen la sintaxis y estructura del lenguaje.

En Python 3.13, hay 35 palabras clave:

```
# Palabras clave en Python 3.13
False      await      else      import     pass
None       break      except    in         raise
True       class      finally  is         return
and        continue  for      lambda     try
as         def        from     nonlocal   while
assert     del        global   not        with
async      elif       if       or         yield
```

Puedes ver la lista completa de palabras clave en tu versión de Python ejecutando:

```
import keyword
print(keyword.kwlist)
```

Las palabras clave se agrupan según su función:

- Valores: `True`, `False`, `None`
- Operadores: `and`, `or`, `not`, `is`, `in`
- Control de flujo: `if`, `elif`, `else`, `for`, `while`, `break`, `continue`, `pass`
- Excepciones: `try`, `except`, `finally`, `raise`
- Definición de estructuras: `def`, `class`, `return`, `yield`, `lambda`
- Ámbito de variables: `global`, `nonlocal`
- Importación: `import`, `from`, `as`
- Contexto: `with`
- Asincronía: `async`, `await`
- Otros: `assert`, `del`

Uso de palabras clave en el código

Veamos ejemplos de cómo se utilizan algunas palabras clave en el código:

```
# Control de flujo
if edad >= 18:
    print("Mayor de edad")
elif edad >= 13:
    print("Adolescente")
else:
    print("Niño")
```

```

# Bucles
for i in range(5):
    if i == 2:
        continue # Salta a la siguiente iteración
    print(i)

# Definición de funciones
def saludar(nombre):
    return f"Hola, {nombre}"

# Manejo de excepciones
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("No se puede dividir por cero")
finally:
    print("Operación finalizada")

# Definición de clases
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

```

Identificadores especiales

Python utiliza convenciones específicas para ciertos tipos de identificadores:

- **Identificadores con doble guion bajo** (`__nombre__`): Métodos especiales o "dunder" (double underscore) que tienen significados predefinidos en Python.
- **Identificadores con guion bajo inicial** (`_nombre`): Indica que un atributo o método es "privado" por convención.
- **Identificadores con doble guion bajo inicial** (`__nombre`): Activa el "name mangling" para evitar colisiones en subclases.

```

class Ejemplo:
    def __init__(self):
        # Método especial (constructor)
        self.publico = 10 # Atributo público
        self._privado = 20 # Atributo "privado" por convención
        self.__protegido = 30 # Atributo con name mangling

    def __str__(self):
        # Método especial (representación de cadena)
        return f"Ejemplo({self.publico})"

```


Buenas prácticas con identificadores

Para escribir código más legible y mantenible:

- Usa nombres **descriptivos** que indiquen el propósito de la variable o función
- Evita nombres de una sola letra excepto en casos específicos (como `i` para índices en bucles)
- Mantén la **consistencia** en el estilo de nomenclatura en todo tu proyecto
- No uses nombres demasiado largos que dificulten la lectura
- Evita nombres similares que puedan causar confusión

```
# Mal ejemplo
a = 5
b = 10
c = a + b

# Buen ejemplo
edad = 5
altura = 10
suma_valores = edad + altura
```

En Visual Studio Code, puedes aprovechar las herramientas de refactorización para renombrar identificadores de manera consistente en todo tu código (clic derecho → Renombrar símbolo o F2).

Verificación de identificadores y palabras clave

Python proporciona funciones para verificar si una cadena es un identificador válido o una palabra clave:

```
# Verificar si una cadena es un identificador válido
nombre = "variable_1"
print(nombre.isidentifier()) # True

# Verificar si una cadena es una palabra clave
import keyword
print(keyword.iskeyword("if")) # True
print(keyword.iskeyword("nombre")) # False
```

Entender los tokens léxicos de Python, especialmente los identificadores y palabras clave, es fundamental para escribir código sintácticamente correcto y seguir las convenciones del lenguaje. Esto no solo hace que tu código sea más legible para ti, sino también para otros desarrolladores que puedan trabajar con él en el futuro.

Comentarios de una línea, multilínea y docstrings

Los comentarios son elementos fundamentales en cualquier código Python que permiten **documentar** y explicar lo que hace el programa. A diferencia de los tokens léxicos que el intérprete procesa, los comentarios son ignorados durante la ejecución y sirven exclusivamente para los programadores.

Python ofrece tres formas principales de incluir comentarios en el código: comentarios de una línea, comentarios multilínea y docstrings. Cada uno tiene propósitos específicos y convenciones de uso que veremos a continuación.

Comentarios de una línea

Los comentarios de una línea comienzan con el símbolo **almohadilla** (`#`) y continúan hasta el final de la línea. Son ideales para explicaciones breves o anotaciones puntuales.

```
# Este es un comentario de una línea
edad = 25 # También puedes colocar comentarios al final de una instrucción
```

Algunos usos comunes de los comentarios de una línea incluyen:

- Explicar el propósito de una variable o expresión
- Aclarar una lógica compleja
- Marcar secciones del código
- Desactivar temporalmente líneas de código (comentar código)

```
# Configuración inicial
MAX_INTENTOS = 3 # Número máximo de intentos permitidos

# Cálculo del promedio
total = sum(valores) # Suma todos los elementos
promedio = total / len(valores) # Divide por la cantidad de elementos

# Esta línea está comentada y no se ejecutará
# print("Esto no se mostrará")
```

En Visual Studio Code, puedes comentar o descomentar rápidamente líneas seleccionadas usando el atajo `Ctrl+/*` (o `Cmd+/*` en Mac).

Comentarios multilínea

Aunque Python no tiene una sintaxis específica para comentarios multilínea como otros lenguajes, existen dos enfoques comunes:

- **Múltiples comentarios de una línea:** Usar el símbolo `#` al inicio de cada línea
- **Cadenas de texto no asignadas:** Usar cadenas de texto triple comilla (`'''` o `"""`) que no se asignan a ninguna variable

```
# Este es un comentario
# que ocupa múltiples líneas
# usando el símbolo # en cada línea

'''
Este es otro enfoque para comentarios multilínea
usando cadenas de texto triple comilla
que no se asignan a ninguna variable
'''

"""
También se pueden usar comillas dobles
para el mismo propósito
"""
```

El segundo método (cadenas no asignadas) es menos común para comentarios regulares y se usa principalmente para docstrings, que veremos a continuación.

Docstrings

Los **docstrings** (cadenas de documentación) son un tipo especial de comentario que se coloca al inicio de un módulo, función, clase o método. A diferencia de los comentarios regulares, los docstrings son retenidos en tiempo de ejecución y pueden ser accedidos mediante el atributo `__doc__`.

Los docstrings se definen usando **triple comillas** (`"""` o `'''`) y siguen convenciones específicas descritas en PEP 257:

```
def calcular_area_circulo(radio):
    """
    Calcula el área de un círculo.

    Args:
        radio (float): El radio del círculo en unidades.

    Returns:
        float: El área del círculo en unidades cuadradas.
```

```

"""
import math
return math.pi * radio ** 2

# Acceder al docstring
print(calcular_area_circulo.__doc__)

```

Los docstrings son especialmente útiles porque:

- Proporcionan **documentación integrada** en el código
- Son accesibles mediante la función `help()` y el atributo `__doc__`
- Son utilizados por herramientas de generación de documentación como Sphinx
- Aparecen en la ayuda contextual de IDEs como Visual Studio Code

Formatos de docstrings

Existen varios estilos de formato para docstrings, siendo los más comunes:

- **Google style:** Usa secciones con encabezados como Args:, Returns:, Raises:
- **reStructuredText (reST):** Usado por Sphinx, con directivas como `:param radio:`
- **NumPy style:** Similar al de Google pero con formato específico para documentación científica
- **Epytext:** Similar a Javadoc, usando etiquetas como @param

El estilo Google es uno de los más legibles y populares:

```

def transferir_fondos(origen, destino, cantidad):
    """Transfiere fondos entre dos cuentas.

    Args:
        origen (Cuenta): La cuenta de origen de los fondos.
        destino (Cuenta): La cuenta de destino de los fondos.
        cantidad (float): La cantidad a transferir.

    Returns:
        bool: True si la transferencia fue exitosa, False en caso contrario.

    Raises:
        ValueError: Si la cantidad es negativa.
        SaldoInsuficienteError: Si la cuenta de origen no tiene fondos suficientes.
    """
    if cantidad < 0:
        raise ValueError("La cantidad no puede ser negativa")

```

```
# Resto del código...
return True
```

Docstrings en clases y módulos

Los docstrings también se utilizan para documentar clases y módulos completos:

```
"""
Módulo de utilidades financieras.

Este módulo proporciona funciones para cálculos financieros
básicos como interés compuesto, amortización de préstamos
y conversión de divisas.
"""

class CuentaBancaria:
    """
    Representa una cuenta bancaria con operaciones básicas.

    Esta clase implementa funcionalidades para depósitos,
    retiros y transferencias entre cuentas.

    Atributes:
        titular (str): Nombre del titular de la cuenta.
        saldo (float): Saldo actual de la cuenta.
        numero (str): Número único de la cuenta.
    """

    def __init__(self, titular, saldo_inicial=0):
        """
        Inicializa una nueva cuenta bancaria.

        Args:
            titular (str): El nombre del titular de la cuenta.
            saldo_inicial (float, optional): El saldo inicial. Por defecto es 0.
        """
        self.titular = titular
        self.saldo = saldo_inicial
        self.numero = self._generar_numero()
```

Buenas prácticas para comentarios

Para mantener un código limpio y bien documentado:

- **Comenta el porqué, no el qué:** El código ya muestra qué hace; los comentarios deben explicar por qué lo hace
- **Mantén los comentarios actualizados:** Comentarios obsoletos son peores que ningún comentario
- **Usa docstrings para toda función/clase pública:** Documenta la interfaz, no la implementación
- **Sé conciso pero completo:** Evita comentarios obvios pero no escatimes en explicaciones necesarias
- **Usa comentarios para marcar código que necesita revisión:** Etiquetas como `# TODO:`, `# FIXME:` o `# NOTE:` son útiles

```
# Mal comentario (obvio)
x = x + 1 # Incrementa x en 1

# Buen comentario (explica el porqué)
x = x + 1 # Compensación por el error de redondeo en el cálculo anterior

# TODO: Refactorizar esta función para mejorar rendimiento
def proceso_lento():
    pass
```

Visual Studio Code reconoce etiquetas como `TODO` y `FIXME` y las resalta, además de permitir buscarlas fácilmente.

Comentarios especiales para el intérprete

Existen algunos comentarios especiales que, aunque son ignorados como código, tienen significado para el intérprete o las herramientas de desarrollo:

- **Shebang:** `#!/usr/bin/env python3` - Indica qué intérprete usar en sistemas Unix
- **Codificación:** `# -*- coding: utf-8 -*-` - Especifica la codificación del archivo
- **Directivas de tipo:** `# type: ignore` - Para herramientas de verificación de tipos
- **Directivas para linters:** `# noqa` - Para ignorar advertencias de herramientas de análisis

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
Este módulo contiene utilidades para procesamiento de texto.
"""

import antiguo_modulo # noqa: F401 - Necesario para compatibilidad
```

```
def funcion_con_tipo(texto: str) -> int: # type: ignore
    # Esta función tiene anotaciones de tipo pero ignoramos verificaciones
    return len(texto)
```

Los comentarios son una parte esencial del desarrollo en Python, no solo para explicar el código a otros desarrolladores, sino también para tu "yo futuro" que revisará el código meses después. Un buen equilibrio entre código autodocumentado (con nombres descriptivos) y comentarios estratégicos hará que tu código sea más mantenible y profesional.
